

Final Project/Exam: C++ Solution for Sustainable Development

Course Code: CC103

Course Name: Computer Programming 2

Program/Year Level: BSIT, 1st Year

Semester/Term: 2nd Semester

1. Technical Specifications & Requirements

I. Functional Requirements (FR)

Functional requirements define the specific actions and behaviors the system must perform.

- **FR1: Automated Data Initialization**

```
--- CONTACT LIST ---
1. Tiongson - 09785436251
2. Marc - 09564378241
3. Richard - 09674563728
4. Lanz - 09453271689
5. Marco - 09674536218
6. Gian - 0967543216
7. Tiongson - 09745362178
8. Feolino - 09563421534
9. Romero - 09675432567
10. Joseph - 09675342516
11. daniel - 09675346789
12. padilla - 09674352143
13. joshua - 09563425132
14. garcia - 09345187649
15. Tabasa - 09347856342
16. sarah - 09564387921
17. henry - 09342176587
18. sandra - 09674352617
19. mark - 09435672198
20. mei - 096734526157
```

- The system can read an external database file "contact.h" (.txt).

- The system can parse a maximum of 20 pre-existing records and instantiate them as objects within an internal collection as Array.

- **FR2: Secure Record Management (CRUD)**

Create: Users must be able to input new data entries that adhere to the system's class structure.

```
===== SAFECIRCLE SYSTEM =====
1. Create Profile
2. View Profile
3. Add Contact
4. View Contacts
5. Search Contact
6. Update Contact
7. Delete Contact
8. Panic Alert
9. Safety Timer
10. Nearest Safe Location
11. Safety Tips
12. Exit
Enter Choice: 3

Enter Contact Name: Marc Romero
Enter Contact Number: 095634256178

Contact Added Successfully!

Press ENTER to continue...
```

Read: The system must provide a "View All" feature and a "Specific Search" feature (e.g., search by ID or Name).

```
===== SAFECIRCLE SYSTEM =====
```

1. Create Profile
2. View Profile
3. Add Contact
4. View Contacts
5. Search Contact
6. Update Contact
7. Delete Contact
8. Panic Alert
9. Safety Timer
10. Nearest Safe Location
11. Safety Tips
12. Exit

Enter Choice: 4

```
--- CONTACT LIST ---
```

1. Marc - 09674532678
2. Marco - 09564378902
3. Lanz - 09453278964
4. Villaroza - 09342178963

Press ENTER to continue...

```
===== SAFECIRCLE SYSTEM =====
```

1. Create Profile
2. View Profile
3. Add Contact
4. View Contacts
5. Search Contact
6. Update Contact
7. Delete Contact
8. Panic Alert
9. Safety Timer
10. Nearest Safe Location
11. Safety Tips
12. Exit

Enter Choice: 5

Enter Contact Name to Search: Lanz

Contact Found!

Lanz - 09453278964

Press ENTER to continue...

Update: Users must be able to select an existing record and modify its specific attributes (e.g., updating the status of a resource).

```
===== SAFECIRCLE SYSTEM =====
1. Create Profile
2. View Profile
3. Add Contact
4. View Contacts
5. Search Contact
6. Update Contact
7. Delete Contact
8. Panic Alert
9. Safety Timer
10. Nearest Safe Location
11. Safety Tips
12. Exit
Enter Choice: 6

--- CONTACT LIST ---
1. Marc Romero - 095637524167
2. Lanz Tabasa - 09563425167

Enter Contact Number to Update: 1
Enter New Name: Marc Romero
Enter New Number: 09564532678

Contact Updated Successfully!

Press ENTER to continue...
```

Delete: The system must allow for the removal of records from the active session.

```
===== SAFECIRCLE SYSTEM =====
1. Create Profile
2. View Profile
3. Add Contact
4. View Contacts
5. Search Contact
6. Update Contact
7. Delete Contact
8. Panic Alert
9. Safety Timer
10. Nearest Safe Location
11. Safety Tips
12. Exit
Enter Choice: 7

--- CONTACT LIST ---
1. Marc Romero - 09564532678
2. Lanz Tabasa - 09563425167

Enter Contact Number to Delete: 1
Are you sure? (Y/N): Y

Contact Deleted Successfully!

Press ENTER to continue...
```

- **FR3: Core Domain Logic (Computation Engine)**

We have safety timer counting 10 seconds, and a user pin that allows the system identify if the user is at risk

```

main.cpp  contact.h  contact.cpp  validation.h  validation.cpp  safety.h  safety.cpp  filehandler.h  filehandler.cpp  contacts.txt
12
13 void safetyTimer(string userPin) {
14
15     int timer = 10;
16     string pin;
17
18     cout << "\nSafety Timer Started...\n";
19
20     while(timer > 0) {
21
22         cout << "Time left: " << timer << " seconds\n";
23
24         this_thread::sleep_for(chrono::seconds(1));
25
26         timer--;
27     }
28
29     cout << "\nTimer Ended!\n";
30
31     cout << "Enter 4-digit PIN to deactivate: ";
32     cin >> pin;
33
34     if(pin == userPin) {
35
36         cout << "\nTimer Deactivated Successfully.\n";
37     }
38
39     else {
40
41         cout << "\nUSER AT RISK!\n";
42     }

```

- **FR4: Persistent State Storage**

The system must synchronize the internal memory with the external file. All additions, deletions, and updates made during the session must be saved to the file upon a clean exit.

```
input
1. Create Profile
2. View Profile
3. Add Contact
4. View Contacts
5. Search Contact
6. Update Contact
7. Delete Contact
8. Panic Alert
9. Safety Timer
10. Nearest Safe Location
11. Safety Tips
12. Exit
Enter Choice: 3

Enter Contact Name: Romero
Enter Contact Number: 09928740918

Contact Added Successfully!

Press ENTER to continue...

===== SAFECIRCLE SYSTEM =====
1. Create Profile
2. View Profile
3. Add Contact
4. View Contacts
5. Search Contact
6. Update Contact
7. Delete Contact
8. Panic Alert
9. Safety Timer
10. Nearest Safe Location
11. Safety Tips
12. Exit
Enter Choice: 12

Program Closed.

Press ENTER to continue...[]
```

```
main.cpp | contact.h | contact.cpp | validation.h | validation.cpp | safety.h | safety.cpp | filehandler.h | filehandler.cpp | contacts.txt
1
2
3 Romero
4 09928740918
5
```

- **FR5: Navigational Menu System**

Enter Choice: 3

Enter Contact Name: Marc Romero
Enter Contact Number: 09564376891

Contact Added Successfully!

Press ENTER to continue...

3

===== SAFECIRCLE SYSTEM =====

1. Create Profile
2. View Profile
3. Add Contact
4. View Contacts
5. Search Contact
6. Update Contact
7. Delete Contact
8. Panic Alert
9. Safety Timer
10. Nearest Safe Location
11. Safety Tips
12. Exit

Enter Choice: 3

Enter Contact Name: Lanz Joesph
Enter Contact Number: 09452367183

Contact Added Successfully!

Press ENTER to continue...

5

===== SAFECIRCLE SYSTEM =====

1. Create Profile
2. View Profile
3. Add Contact
4. View Contacts
5. Search Contact
6. Update Contact
7. Delete Contact
8. Panic Alert
9. Safety Timer
10. Nearest Safe Location
11. Safety Tips
12. Exit

Enter Choice: 5

Enter Contact Name to Search: Marc Romero

Contact Found!

Marc Romero - 09564376891

Press ENTER to continue...

```
Enter Contact Name to Search: Marc Romero
```

```
Contact Found!
```

```
Marc Romero - 09564376891
```

```
Press ENTER to continue...
```

```
===== SAFECIRCLE SYSTEM =====
```

1. Create Profile
2. View Profile
3. Add Contact
4. View Contacts
5. Search Contact
6. Update Contact
7. Delete Contact
8. Panic Alert
9. Safety Timer
10. Nearest Safe Location
11. Safety Tips
12. Exit

```
Enter Choice: 7
```

```
--- CONTACT LIST ---
```

1. Marc Romero - 09564376891
2. Lanz Joesph - 09452367183

```
Enter Contact Number to Delete: 2
```

```
Are you sure? (Y/N): Y
```

```
Contact Deleted Successfully!
```

```
Press ENTER to continue...
```

A robust, looped menu interface must allow users to transition between modules (Search, Add, Report, Save) without program termination.

II. Non-Functional Requirements (NFR)

Non-functional requirements define the quality constraints and professional standards of the software.

- **NFR1: Robustness & Defensive Programming**

Input Validation: The system must handle "Type Mismatch" errors (e.g., entering a letter when a number is expected) without crashing or entering infinite loops.

Graceful Failure: Informative error messages must guide the user when invalid operations are attempted.

```
===== SAFECIRCLE SYSTEM =====
1. Create Profile
2. View Profile
3. Add Contact
4. View Contacts
5. Search Contact
6. Update Contact
7. Delete Contact
8. Panic Alert
9. Safety Timer
10. Nearest Safe Location
11. Safety Tips
12. Exit
Enter Choice: 3

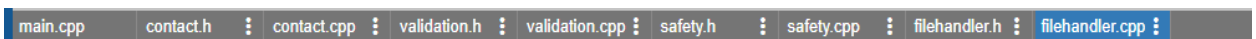
Enter Contact Name: Arwehn Tiongson
Enter Contact Number: 78f97fs
Invalid Number. Enter Digits Only: 097th8990
Invalid Number. Enter Digits Only: 09282790962

Contact Added Successfully!

Press ENTER to continue...
```

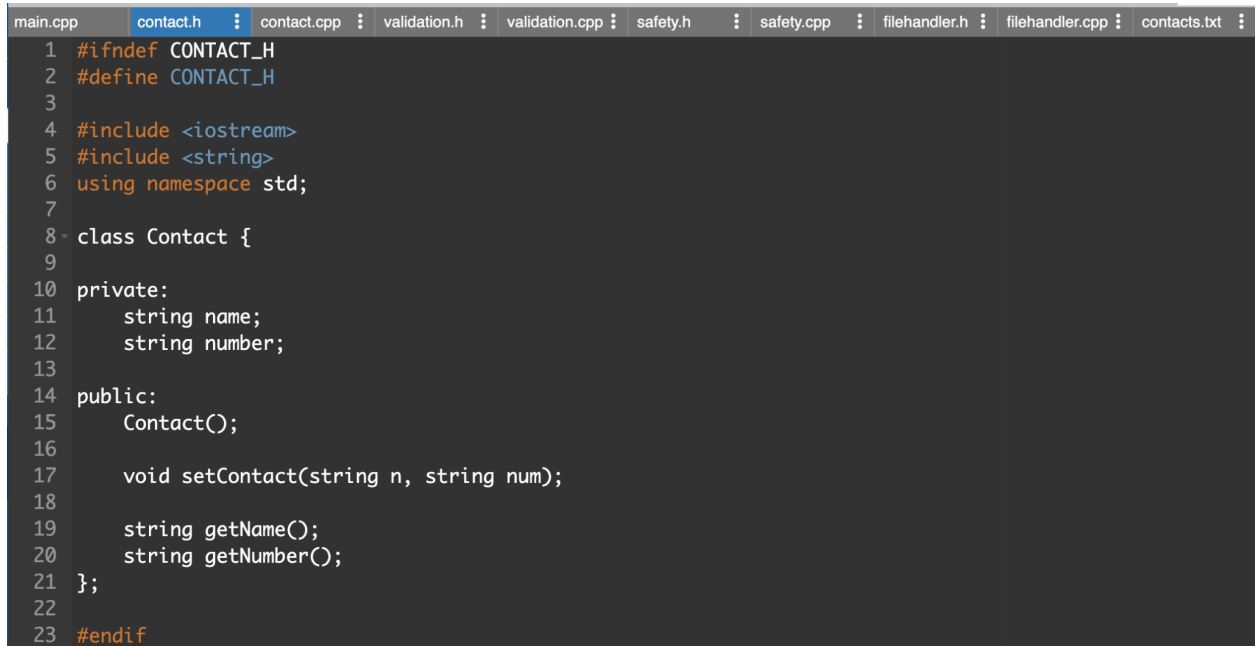
- **NFR2: Structural Modularity**

Multi-File Separation: The project must be partitioned into `.h` files (declarations) and `.cpp` files (implementations) to demonstrate professional code organization.



A screenshot of a code editor's tab bar showing a project structure. The tabs are: `main.cpp`, `contact.h`, `contact.cpp`, `validation.h`, `validation.cpp`, `safety.h`, `safety.cpp`, `filehandler.h`, and `filehandler.cpp`. The `filehandler.cpp` tab is currently selected and highlighted in blue.

Encapsulation: All class attributes must be **private**, accessed only via **public** getter and setter methods.

A screenshot of a code editor with a dark theme. The editor has a tab bar at the top with several files: main.cpp, contact.h (selected), contact.cpp, validation.h, validation.cpp, safety.h, safety.cpp, filehandler.h, filehandler.cpp, and contacts.txt. The main editing area shows the content of contact.h, which is a C++ header file. It starts with a guard to prevent multiple inclusions, followed by includes for <iostream> and <string>, and a using namespace std; statement. Then, a class named Contact is defined. Inside the class, there is a private section with two member variables: string name; and string number;. There is a public section with a default constructor Contact();, a setter method void setContact(string n, string num);, and two getter methods string getName(); and string getNumber();. The class definition ends with a closing brace and semicolon. The file guard ends with #endif.

```
1 #ifndef CONTACT_H
2 #define CONTACT_H
3
4 #include <iostream>
5 #include <string>
6 using namespace std;
7
8 class Contact {
9
10 private:
11     string name;
12     string number;
13
14 public:
15     Contact();
16
17     void setContact(string n, string num);
18
19     string getName();
20     string getNumber();
21 };
22
23 #endif
```

- **NFR3: Readability & Professional Documentation**

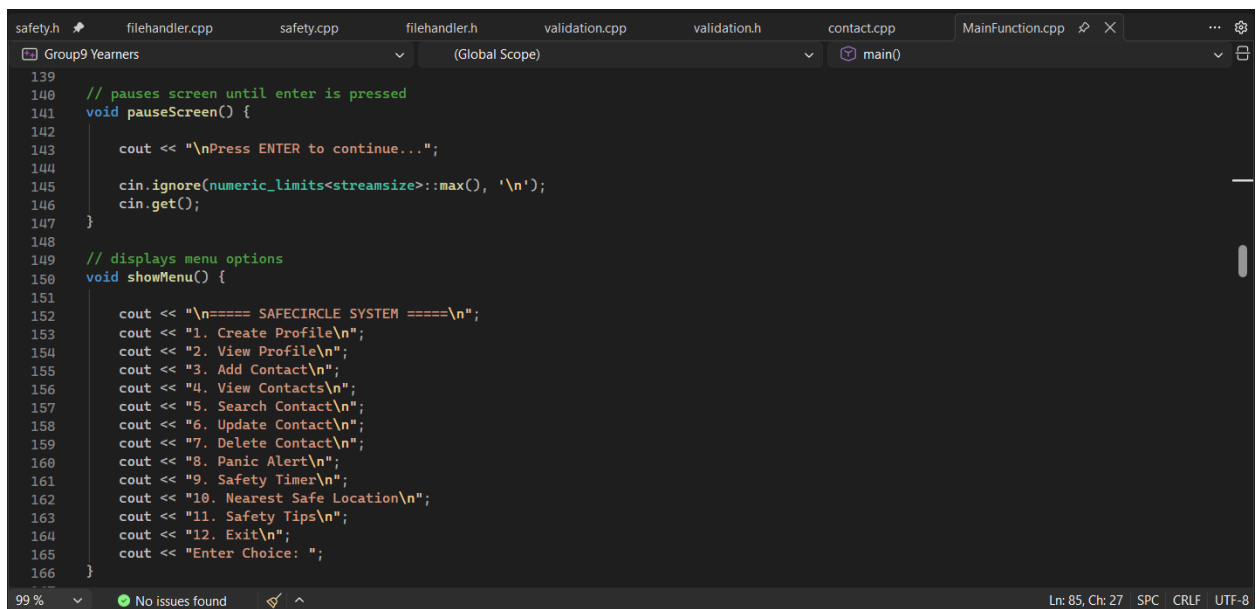
Naming Conventions: We used camelCase for our variables and functions names.

```

19
20 void showMenu();
21 void createProfile();
22 void viewProfile();
23 void addContact(int &count);
24 void viewContacts(int count);
25 void searchContact(int count);
26 void updateContact(int count);
27 void deleteContact(int &count);
28 void pauseScreen();
29

```

In-Code Documentation: Every class and function must be preceded by a comment block detailing its purpose, inputs, and outputs.



```

safety.h  filehandler.cpp  safety.cpp  filehandler.h  validation.cpp  validation.h  contact.cpp  MainFunction.cpp
Group9 Yearmers  (Global Scope)  main()
139
140 // pauses screen until enter is pressed
141 void pauseScreen() {
142
143     cout << "\nPress ENTER to continue...";
144
145     cin.ignore(numeric_limits<streamsize>::max(), '\n');
146     cin.get();
147 }
148
149 // displays menu options
150 void showMenu() {
151
152     cout << "\n==== SAFECIRCLE SYSTEM =====\n";
153     cout << "1. Create Profile\n";
154     cout << "2. View Profile\n";
155     cout << "3. Add Contact\n";
156     cout << "4. View Contacts\n";
157     cout << "5. Search Contact\n";
158     cout << "6. Update Contact\n";
159     cout << "7. Delete Contact\n";
160     cout << "8. Panic Alert\n";
161     cout << "9. Safety Timer\n";
162     cout << "10. Nearest Safe Location\n";
163     cout << "11. Safety Tips\n";
164     cout << "12. Exit\n";
165     cout << "Enter Choice: ";
166 }
99 %  No issues found  Ln: 85, Ch: 27  SPC  CRLF  UTF-8

```

```
#include <iostream>
#include <string>
#include <limits>
#include "contact.h"
#include "validation.h"
#include "safety.h"
#include "filehandler.h"

using namespace std;

const int MAX = 20;

string userName;
int userAge;
string userPin;

Contact contacts[MAX];
int contactCount = 0;

void showMenu();
void createProfile();
void viewProfile();
void addContact(int &count);
void viewContacts(int count);
void searchContact(int count);
void updateContact(int count);
void deleteContact(int &count);
void pauseScreen();
```

```
#ifndef CONTACT_H
#define CONTACT_H

#include <iostream>
#include <string>
using namespace std;

class Contact {

private:
    string name;
    string number;

public:
    Contact();

    void setContact(string n, string num);

    string getName();
    string getNumber();
};

#endif
```

```
#ifndef VALIDATION_H
#define VALIDATION_H

#include <iostream>
#include <string>
using namespace std;

bool isNumber(string text);
bool isValidPin(string pin);
bool isValidName(string name);

#endif
```

```
#ifndef SAFETY_H
#define SAFETY_H

#include <iostream>
#include <string>
using namespace std;

void panicAlert();
void safetyTimer(string userPin);
void nearestSafeLocation();
void showSafetyTips();

#endif
```

```
#ifndef FILEHANDLER_H
#define FILEHANDLER_H

#include "contact.h"

void saveContacts(Contact contacts[], int count);
void loadContacts(Contact contacts[], int &count);

#endif
```

- **NFR4: Memory & Resource Efficiency**

- 1. Type Selection**

The goal is to choose the data type that uses the least amount of memory while maintaining necessary precision.

- **For Counters:** Use `int` (or even `short` or `byte` depending on the range) instead of larger types like `long` if the value won't exceed a certain threshold.
- **For Currency/Decimals:** While the image mentions `float/double`, note that in many financial systems, `BigDecimal` or `long` (representing cents) is preferred for precision. However, for raw memory efficiency in high-speed calculations, `float` (4 bytes) is used over `double` (8 bytes) when high precision isn't critical.

2. Scope Management

This requires developers to control the "lifetime" of a variable to free up memory as soon as it is no longer needed.

- **Narrow Scope:** Declare variables inside the specific block (like an `if` statement or a `for` loop) where they are used, rather than at the top of a function or as a global class member.
- **Efficient Lifecycle:** By keeping the scope narrow, the system (or Garbage Collector in languages like Java/C#) can reclaim that memory immediately after the execution leaves that specific block, preventing memory bloat.

2. Software Design & Analysis Document (SDAD)

1.1 Project Overview & Target SDG

SafeCircle is a console-based personal safety management system developed in C++. It allows users to create a personal profile, manage emergency contacts, activate safety features such as a panic alert and a countdown safety timer, locate nearby safe zones, and view curated safety tips. All contact data is persisted across sessions via `contact.txt` file.

SafeCircle is designed in direct alignment with SDG 5 – Gender Equality, particularly Target 5, which aims to eliminate all forms of violence against women and girls in public and private spaces, and Target 5, which calls for the adoption and strengthening of policies that promote gender equality and the protection of women. By providing individuals, especially women and girls with immediate, accessible safety tools, SafeCircle empowers vulnerable persons to seek help quickly, promotes personal security, and contributes to a culture where gender-based violence is actively prevented and reported.

1.2 Problem Statement

Personal safety remains a pressing concern in densely populated urban areas, particularly for commuters, students, and individuals traveling alone. In many situations, a person in distress may not have time or the means to navigate complex applications or place a phone call. There is a need for a lightweight, always-available safety tool that provides one-touch panic alerting to pre-stored emergency contacts. Offers a

timer-based safety check-in that flags the user as at-risk if they fail to deactivate it with their PIN.

Guides the user to the nearest categorised safe location (police station, hospital, barangay hall, or convenience store). Stores and manages emergency contacts reliably between sessions. SafeCircle addresses these needs through a simple numbered-menu interface executable on any terminal, requiring no internet connectivity and no special hardware beyond a standard computer.

Section 2: Requirements Analysis

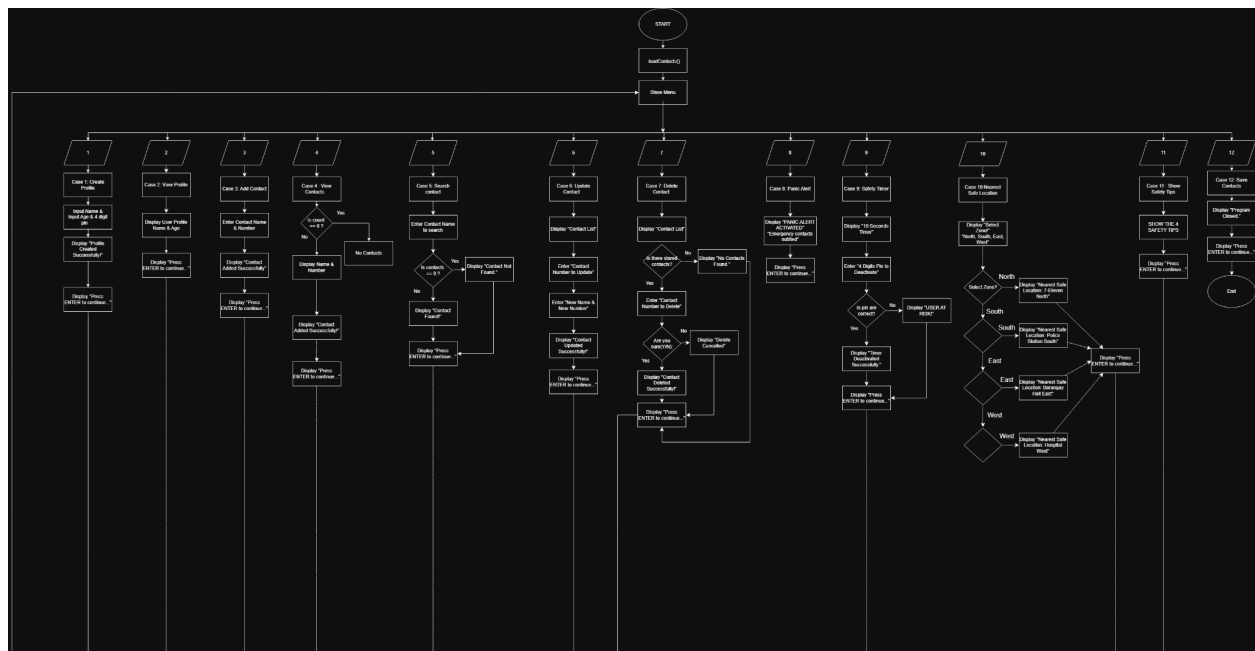
2.1 Functional Requirements (FR) The table below lists the functional requirements derived directly from the implemented source code.

ID	Requirement Description	Source
FR1	System must allow the user to create a personal profile (name, age, 4-digit PIN).	createProfile()
FR2	System must display the stored user profile on demand.	viewProfile()
FR3	System must allow adding an emergency contact (name + phone number) up to a maximum of 20 entries.	addContact()
FR4	System must display all stored contacts in a numbered list.	viewContacts()
FR5	System must support case-sensitive name-based contact search.	searchContact()
FR6	System must allow updating any contact's name and number by selecting its list index.	updateContact()
FR7	System must allow deleting a contact after confirmation (Y/N) and shift remaining entries.	deleteContact()
FR8	System must trigger a panic alert message simulating notification of emergency contacts.	panicAlert()

FR9	System must run a 10-second safety timer; if the user fails to enter the correct PIN, display 'USER AT RISK!'.	safetyTimer()
FR10	System must guide the user to a nearest safe location based on a selected compass zone (N/S/E/W).	nearestSafeLocation()
FR11	System must display a list of four personal safety tips.	showSafetyTips()
FR12	System must persist contacts to contacts.txt on exit and reload them automatically at startup.	saveContacts() / loadContacts()

Section 3: Design Specifications

- 3.1 **Algorithmic Flowchart:** A detailed flowchart of the system's most complex logic.



- 3.2 **File Structure Diagram:** Explanation of the data format used in external files.

Contacts.txt

This is our database file which contains all stored emergency contact records from the user.

SafeCircle Program automatically works:

- **reads the file during startup**
- **loads the contacts into the array of objects**
- **saves updated records before exiting**

ifstream used for reading the file, while **getline(file, name)** are used for getting the name and the number of the user.

contacts[count].setContact(name, number); is used for putting the data of the user to arrays of the object **Contact contacts[MAX];** the use of this code is to limit the number of emergency contacts on our program
const int MAX = 20;

3. References

https://zhjwpku.com/assets/pdf/books/C++.Primer.5th.Edition_2013.pdf

Authors: Stanley B. Lippman, Josée Lajoie, Barbara E. Moo

Used for:

- classes
- encapsulation
- constructors
- Validation

<https://cplusplus.com/>

Used for:

- C++ syntax

- loops
- arrays
- file handling
- string functions

<https://www.w3schools.com/cpp/>

Used for:

- beginner C++ examples
- switch-case
- loops
- validation

<https://www.geeksforgeeks.org/cpp/c-plus-plus/>

Used for:

- object-oriented programming
- arrays
- CRUD operations
- file handling examples